REFLECTIVE REPORT

Z02DT – SOFTWARE AND DATABASES

Student Id: 16873454
Student Name: Zhanel Assilbek

Uncontrolled if printed

TABLE OF CONTENTS

List of Illustrations 2

List of TABLES 3

Introduction 4

Findings and Analysis 4

Software Development LifeCycle (SDLC) 4

Requirement Elicitation and ENGINEERING 4

Programming Constructs 5

TESTING 5

Object Oriented Programming (OOP) Principles 6

Database Design 6

Database Design (ERD AND NORMALISATION) 6

Database Design (SQL) 7

Consideration of the ethical issues in software design and data storage 7

Conclusion and Areas of Further improvement 7

AI Use referencing 8

List of References 8

Appendix 1 – Requirement Catalogue USING FURPS & MOSCOW 10

Appendix 2 - PYTHON Code Samples Screenshots of PROGRAMMING CONSTRUCTs 12

Appendix 3 - PYTHON Code Samples Screenshots of OOP 15

Appendix 4 - Entity Relationship Diagrams (Conceptual, logical, and physical) 16

Appendix 5 – SAMPLES OF DDL/DML SQL STATEMENTS USED 17

Appendix 6 – Test logs 21

Appendix 7 – OUTPUT Screenshots 22

LIST OF ILLUSTRATIONS

Figure 1- Linear Execution 13

Figure 2 - Selection in set_grade() 13

Figure 3 - Iteration in attendance_report() 14

Figure 4 - Python code that I am proud of 14

Figure 5 - StudentInfo Inheritance from User class 15

Figure 6 - TeacherInfo Inheritance from User class 15

Figure 7 - Conceptual ERD 16

Figure 8 - Logical ERD 16

Figure 9 - Physical ERD 17

Figure 10 - CREATE TABLE Students statement and database index optimization 17

Figure 11 - CREATE TABLE Registrations statement 18

Figure 12 - CREATE TABLE Attendance statement 18

Figure 13 - Stored procedure to get registration report 18

Figure 14 - Stored Procedures for data Insertion and modification 19

Figure 15 - Stored Procedure to get attendance report 19

Figure 16 - CALL statements from testing.sql 20

Figure 17 - Registration report in SQL database 20

Figure 18 - Main menu 22

Figure 19 - Adding student with valid data 22

Figure 20 - Invalid ID testing 22

Figure 21 - Invalid email testing 22

Figure 22 - Single-letter name testing 23

Figure 23 - Invalid grade 23

Figure 24 - Not existing ID testing 23

Figure 25 - Deleting student by ID 23

Figure 26 - Registration report 24

Figure 27 - Student Attendance Report 24

LIST OF TABLES

Table 1: Weekly Deadlines 10

Table 2: GANTT Chart 10

Table 3: MoSCoW prioritized Backlog 10

Table 4: FURPS 11

Table 5: Python readability principles 12

Table 6: Python Variable types 12

Table 7: Test Logs 21

INTRODUCTION

This report reflects on the process of designing a school management system. My project includes Python script and the MySQL database. It is aimed to help with tasks such as registering students and marking attendance. I designed the system that can update, delete and find data. Initially, the system had four entities: Students, Teachers, Courses and Registration. Later, I decided to add Attendance table and Grade attribute in the Registrations table to make the system more functional. To fill the database and test my system, I imported CSV file with fictional data. Also, I integrated bubble sort to order Registrations by ID. In this report, I reflected on the whole process of developing my project starting from SDLC choice and finishing with database design justifications.

FINDINGS AND ANALYSIS

SOFTWARE DEVELOPMENT LIFECYCLE (SDLC)

The SDLC divides software development into specific steps (Hossain, 2023). I used an Iterative model. I did not choose Waterfall because of change limitations and linear structure (Pargaonkar, 2023). Agile did not suit because it needs constant team collaboration and client's feedback, while I do not have a team and real clients (Pargaonkar, 2023). So, breaking the project development into small steps was better option for me. I decided to spend three weeks only on artifact and remaining weeks on the report. This planning helped me to meet the final deadline. Each week of building the artifact consisted small iterations that I had to finish. In addition, after each change I did testing and debugging if was needed. Although the Iterative model has simplified the process by dividing it into smaller tasks, it has expanded development time (Hossain, 2023). The GANTT chart and Weekly deadlines table are in Appendix 1.

REQUIREMENT ELICITATION AND ENGINEERING

Requirement elicitation is when the developer understands the client's requests (Laplante & Kassab, 2022). Engineering them is organizing them into the most and least important. I worked individually with no real clients, so the assignment brief and codebase specifications become my clients. I analyzed the requirements from these documents. Then, I divided them into functional and non-functional. This was effective, because the documents clearly define what is mandatory and optional. Furthermore, I used FURPS and MoSCoW to visually organize requirements. Functionality, Usability, Reliability and Supportability categories helped me understand what each requirement ensures (Laplante & Kassab, 2022). MoSCoW aid me in prioritizing Backlog task list (Model & Herzwurm, 2022). Must Have requirements were iterations for first two weeks, while optional GUI placed in Won't Have. These schemas kept me focused only on essential tasks and moving step-by-step. The MoSCoW prioritized backlog and FURPS tables are in Appendix 1.

PROGRAMMING CONSTRUCTS

I applied python principles of readability by giving meaningful names and doing indentations. Also, I used single-responsibility functions by writing for most methods only one CRUD operation, but the show_report() and attendance_report() can also present data. I tried to give variables names that match which their purpose. For example, s_name for student name and reg_id for registration ID. For complex code lines, such as bubble sort, I wrote comments that explain the steps. Moreover, I used several constructs. Firstly, conditional statements were used in different files for specific goals. In main.py file, for processing each menu choices depending on each number and letter. In models.py, for validating name and email format for students and teachers, while in db_controller.py, for validating grades. Secondly, iterations were used in the for loop to import data from the CSV and for loop from to run bubble sorting. Thirdly, try-except statements were used to protect the system from crashing. In db_controller.py, they catch mysql.connector.Error and ValueError and for each error system returns detailed message.

The code I am most proud of is import_from_csv() in import_data.py. This is because it imports data to Students and Teachers tables based on prefixes from the CSV file. Dr. and Mrs. prefixes correspond to teachers, while Mr. and Ms. correspond to students. Also, I wrote code lines that skip empty rows. In addition, jobTitle column in fictional data is used for Department attribute in Teachers table. Also, this code manages duplicate entries by using errorcode.ER_DUP_ENTRY. The challenging part was to ensure that all other database errors are handled, except for duplicate ones. Screenshots and tables are in Appendix 2.

TESTING

Software testing is checking the system for functionality and bugs (Kumar, 2023). I used manual and integration testing. I did manual testing by running main.py and choosing each menu option. I tested with valid and invalid inputs to check error handling. The examples of it are tests from T02 to T05 in Table 7. Manual testing was practical because the system does not have GUI (Thant & Tin, 2023). Integration testing was done by executing testing.sql. This script truncates tables and fetches data. More detailed information about this test is in T08 and T10. The testing confirmed that all tables are connected through foreign keys and stored procedures are written correctly. These types of testing were enough for this project, so I did not use automated frameworks. However automated tests save time (Thant & Tin, 2023). Most of the time I spent on retesting validation methods, so next time I will write unit tests for models.py. Test logs are in Appendix 6.

OBJECT ORIENTED PROGRAMMING (OOP) PRINCIPLES

OOP structures code into objects and classes that have common behavior (Sheta, 2022). Since objects can represent real-world entities, this simplifies data management in Python (AlShammari, 2025). I applied three OOP principles in models.py file. First, my StudentInfo and TeacherInfo child classes inherit from User parent class. So, they receive all methods and properties of it through super() and __init__(). Second, I used encapsulation by making sure that each class has its own attributes and validation method (Sheta, 2022). Third, polymorphism demonstrated though is_valid() method that is written in both child classes but has distinct rules for each. The method is further called inside add_student() and add_teacher() in db_controller.py to add valid name and email to the database. It removes the need to write the same lines in several parts of code (AlShammari, 2025). OOP made my job easier by removing the need to fix the same code lines in different parts of code. So, I if I faced some issues with validation, I could just modify method in StudentInfo and TeacherInfo classes. Screenshots of models.py are in Appendix 3.

DATABASE DESIGN

DATABASE DESIGN (ERD AND NORMALISATION)

My DBMS is MySQL, which is accessible tool for relational data management. It works through connecting foreign keys and creating stored procedures (Wahyudi et al., 2022). In comparison with SQLite, MySQL is more suitable for multi-user environments and extensive databases (Erike et al., 2025). For commercial use of the system, I would recommend using a cloud MySQL database and user authentication to prevent unauthorized access.

An ERD is a diagram that shows entities, their attributes and relationships. I designed three types. The Conceptual ERD demonstrates the main relationships between entities. For example, one Student enrolls in many Courses. Detailed relationships are in Figure 7. The Logical ERD adds primary and foreign keys information, as you can see on Figure 8. The Physical ERD shows data types from database.sql. For example, VARCHAR(100) for names (Figure 9). All three ERDs are available in Appendix 4.

Normalization reduces redundancy and improves performance of the system (Hardini et al., 2025). I achieved 3NF by following several rules. First, I ensured that my tables consist only atomic values to establish 1NF. Second, my system satisfies 2NF because every non-key attribute depends on the full primary key. Finally, I guaranteed that transitive dependencies are eliminated to satisfy 3NF.

DATABASE DESIGN (SQL)

SQL is the standard language for managing relational databases (Wahyudi et al., 2022). All DDL statements were used in the database.sql. Data types were chosen depending on what each column stores. For example, INT for IDs and DATE for dates. Other examples are available in Table 6. Constraints include NOT NULL, UNIQUE on emails, CHECK on Grade and AttendanceStatus, Credits > 0, and ON DELETE CASCADE on all foreign keys. I used stored procedures to implement all CRUD operations including update and search. For instance, sp_UpdateStudent and sp_FindStudent. I integrated them not only for students, but also for teachers and courses. I stored all SQL strings inside stored procedures, so I did not write long SQl statements in the Python (Uzzaman et al., 2024). DDL statements' screenshots and table are in Appendix 5.

CONSIDERATION OF THE ETHICAL ISSUES IN SOFTWARE DESIGN AND DATA STORAGE

The system stores personal names and emails. GDPR requires a specific purpose for data collection and protection measures. I used fictional data, but the design still has risks. I hardcoded the root password in db_controller.py. This exposes credentials and reduces security level. A safer way is to store credentials in environment variables. Access control is also crucial. Admin roles should control who can see and edit private data. If the system handled real students, it would need a clear policies and deletion rules related to data. So, the current design has some gaps. I would fix them before real deployment.

CONCLUSION AND AREAS OF FURTHER IMPROVEMENT

I built a school management system with Python and MySQL. I created five tables and used stored procedures to manage data. I connected database to Python and tested with a CSV import. Some design choices need improvement. The SchoolDataControl class handles too many tasks. I would split it into smaller classes for each entity. This would improve structure. I would also move the into environment variable for higher security. I relied on manual testing during development. This slowed progress. Next time, I would write unit tests for models.py and automate testing.

During this module, I learned how relational databases work. I practiced normalization and stored procedures. I faced type mismatch issues during Python and MySQL integration. Solving these problems improved my confidence. The database is functional but not ready for real data. It lacks encryption and strict access control. I should work on user authentication and GUI.

AI USE REFERENCING

AI Use Category	Tool	How AI Tool was used in this assignment
Learning support	Gemini	to clarify complex SQL syntax for stored procedure sp_GetAllRegistrations
Advising	Gemini	to get advice and explanations for resolving the errors
Proofreading	Gemini	to check grammar and improve language

LIST OF REFERENCES

Alshammari, A. (2025). Implementation of Data Management using Object-Oriented Programming (OOP) in Python. *International Journal of Computer Applications*, *187*, 54-61. https://www.ijcaonline.org/archives/volume187/number49/alshammari-2025-ijca-925844.pdf

Erike, A. I., Alameziem, M. C. C., Elei, F. O., Ovwonuri, A. O., & Nwandu, I. C. (2025). A Comparative Performance Evaluation of SQLite, MySQL, and Firebase for Modern Application Development Using a Parallel Execution Approach. *UNIZIK Journal of Engineering and Applied Sciences*, *5*(4), 2933-2944. https://journals.unizik.edu.ng/ujeas/article/view/7319

Hardini, M., Agarwal, V., Apriani, D., Widjaya, I. A., Setiawaty, E., & Nurasiah, N. (2025). Application of database normalization in increasing data storage efficiency. *International Transactions on Artificial Intelligence*, *3*(2), 201-211. https://doi.org/10.33050/italic.v3i2.799

Hossain, M. I. (2023). Software development life cycle (SDLC) methodologies for information systems project management. *International Journal For Multidisciplinary Research*, *5*(5), 1-36. https://pdfs.semanticscholar.org/9eed/fc508509d415c305116ffb258ff5147fd8b8.pdf

Kumar, S. (2023). Reviewing software testing models and optimization techniques: an analysis of efficiency and advancement needs. *Journal of Computers, Mechanical and Management*, *2*(1), 32-46. https://doi.org/10.57159/gadl.jcmm.2.1.23041

Laplante, P. A., & Kassab, M. (2022). *Requirements engineering for software and systems*. Auerbach Publications. https://doi.org/10.1201/9781003129509

Model, K., & Herzwurm, G. (2022, November). Software-Supported Product Backlog Prioritization in Scrum Software Development Projects. *ICSOB Companion*. https://ceur-ws.org/Vol-3316/phd-paper2.pdf

Pargaonkar, S. (2023). A comprehensive research analysis of software development life cycle (SDLC) agile & waterfall model advantages, disadvantages, and application suitability

in software quality engineering. *International Journal of Scientific and Research Publications*, *13*(8), 120-124. http://dx.doi.org/10.29322/IJSRP.13.08.2023.p14015

Sheta, S. V. (2022). An Overview of Object-Oriented Programming (OOP) and Its Impact on Software Design. *Educational Administration: Theory and Practice*, *28*(4), 409-419. https://doi.org/10.53555/kuey.v28i4.8473

Thant, K. S., & Tin, H. H. K. (2023). The impact of manual and automatic testing on software testing efficiency and effectiveness. *Indian journal of science and research*, *3*(3), 88-93. https://www.ijsronline.org/issue/20230714-032703.942.pdf

Uzzaman, A., Jim, M. M. I., Nishat, N., & Nahar, J. (2024). Optimizing SQL databases for big data workloads: techniques and best practices. *Academic Journal on Business Administration, Innovation & Sustainability*, *4*(3), 15-29. https://www.researchgate.net/publication/381725561_OPTIMIZING_SQL_DATABASES_FORBIG_DATA_WORKLOADS_TECHNIQUES_AND_BEST_PRACTICES

Wahyudi, J., Asbari, M., Sasono, I., Pramono, T., & Novitasari, D. (2022). Database management in MySQL. *Edumaspul-J. Pendidik*, *6*(2), 2413-2417. https://www.academia.edu/104384030/Database_Management_Education_in_MYSQL

APPENDIX 1 – REQUIREMENT CATALOGUE USING FURPS & MOSCOW

Table 1: Weekly Deadlines

Week	Completed tasks
1	Artefact: Drew the ERD; designed the database schema in MySQL; wrote DDL scripts to create tables
2	Artefact: Wrote MySQL Stored procedures for CRUD; set up connection to Python database controller code; added OOP models (classes for teachers and students)
3	Artefact: Imported CSV data; Executed testing.sql file and fixed the NULL bugs; completed artefact
4	Report: wrote SDLC, requirements, programming constructs sections
5	Report: wrote OOP, Database design, testing and data ethics sections; did proofreading, and prepared for submission

Table 2: GANTT Chart

Task	Week 1	Week 2	Week 3	Week 4	Week 5
ERD and scheme design					
Stored Procedures (SQL)					
Python controller and models					
CSV import script					
Integration testing and bug fixes					
Finished Artefact					
Report writing					
Proofreading and submission					

Table 3: MoSCoW prioritized Backlog

Priority	Task
Must Have	Create Students, Teachers, Courses, Registration tables
Must Have	Implement full CRUD using stored procedures
Must Have	Connect Python to MySQL using mysql.connector
Must Have	Input validation in models.py (StudentInfo, TeacherInfo)
Must Have	Interactive terminal menu in main.py
Should Have	Attendance table and mark attendance functionality
Should Have	Grade attribute and set_grade procedure
Should Have	CSV data import via import_data.py
Could Have	Bubble sort on registration report output by RegID in ascending order.
Could Have	TRUNCATE with FOREIGN_KEY_CHECKS = 0 in testing.sql
Won't Have	GUI
Won't Have	Web-based front end

Table 4: FURPS

Category	Requirement description
Functionality	CRUD for students, teachers, and courses. The system should mark attendance, set grades, and sort the registration reports by RegID. It also reads university CSV file and loads data into the tables in database
Usability	The terminal menu must be user-friendly
Reliability	Foreign keys keep data consistent when records get deleted. Python uses try-except statements to prevent crashing.
Performance	The database connections and terminal queries should load quickly and without delays
Supportability	The Python code must be clean and split into separate files

APPENDIX 2 - PYTHON CODE SAMPLES SCREENSHOTS OF PROGRAMMING CONSTRUCTS

Table 5: Python readability principles

Principle	Where Applied	Example
Meaningful variable names	Throughout all files	clean_grade, reg_id, a_status
Single-responsibility functions	db_controller.py	Each method handles one CRUD operation
Inline comments	db_controller.py, import_data.py	Comments explain logic
Consistent indentation	All files	indentation applied across all methods
Input sanitization before use	models.py	.strip() and .capitalize() called before validation runs

Table 6: Python Variable types

Variable Type	Example from code	Use in project	Limitation
int	s_id, c_id, t_id	Stores IDs passed to stored procedures	Cannot store NULL; causes ValueError if non-numeric input given
str	clean_grade, a_status, s_name	Stores names, emails, grades, statuses	No automatic length validation, oversized strings only caught by MySQL
bool	return value of is_valid()	Controls whether database call proceeds	Only True/False; does not describe which specific validation failed
list	data in show_report()	Stores all registration rows for sorting and printing	Entire result set loaded into memory at once
None	formatted_date in register_student()	Passes NULL to MySQL so procedure defaults to CURDATE()	Requires explicit None check; easy to accidentally pass empty string instead
dict	row in import_from_csv()	Map CSV column headers to values via DictReader	Keys depend on exact CSV header names; any mismatch causes a KeyError

Python Constructs:

```python
#main class that manages database and connections
class SchoolDataControl:  4 usages
    def __init__(self):
        # connecting to mysql
        try:
            self.db_link = mysql.connector.connect(
                host='localhost',
                user='root',
                password='Qwerty1!', #password of my local mysql
                database='SchoolManagementSystem'
            )
            self.cursor = self.db_link.cursor()
            print("Successfully linked to database.")
        except Error as e:
            #if the database connection fails, print error message
            print("Connection error", e)
```

Figure 1- Linear Execution

This code runs line by line to connect Python to my local MySQL database using mysql.connector.

```python
    # creates operation to set grade
    def set_grade(self, reg_id, grade):  1 usage
        #grade validation to comply with CHECH Constraint in SQL
        valid_grades = ["A", "B", "C", "D", "F", "N/A"]
        clean_grade = grade.strip().upper()
        if clean_grade not in valid_grades:
            print(f"Validation error: Grade '{clean_grade}' is invalid. Choose from : {', '.join(valid_grades)}")
            return

        try:
            self.cursor.callproc( procname: 'sp_SetGrade',  args: [int(reg_id), clean_grade])
            self.db_link.commit()
            print(f"Grade '{grade}' was successfully set for Registration ID {reg_id}.")
        except ValueError:
            print("Registration ID must be an integer.")
        except Error as e:
            print("Error while setting grade:", e)
```

Figure 2 - Selection in set_grade()

This is grade validation in set_grade(). The if statement checks whether the input matches a list of valid grades. If the grade is not in the list, the method stops and prints an error. The database does not receive anything.

```python
#creates operation to show attendance information
def attendance_report(self):  1 usage
    try:
        self.cursor.callproc('sp_GetAllAttendanceReport')
        for r in self.cursor.stored_results():
            data = r.fetchall()
            print("\n Student Attendance Report")
            print(f"{'Student Name':<25} | {'Course':<15} | {'Status':<10}")
            print("-" * 55)
            if not data:
                print("No attendance records found.")
            else:
                for d in data:
                    print(f"{d[0]:<25} | {d[1]:<15} | {d[2]:<10}")
    except Error as e:
        print("Could not load report:", e)
```

Figure 3 - Iteration in attendance_report()

The for loop iterates through the rows returned by stored procedure and outputs each one to a formatted table.

```python
#creates operation to show registration information
def show_report(self):  1 usage
    # get all data from the procedure results
    try:
        self.cursor.callproc('sp_GetAllRegistrations')
        for r in self.cursor.stored_results():
            data = list(r.fetchall()) #turning into list to sort
            #bubble sort
            n = len(data) #gets the total number of records in the list
            for i in range(n):
                for j in range(0, n-i-1):
                    #data[j][0] is the RegID. checks if the current is bigger than next
                    if data [j][0] > data[j+1][0]:
                        #if yes, swaps
                        data[j], data[j+1] = data[j+1], data[j]

            print("\n Registration Report")
            print(f"{'RegID':<6} | {'Student':<20} | {'Course':<20} | {'Teacher':<15} | {'Date':<12} | {'Grade'}")
            print("-" * 90)
            #loop to print each sorted row
            for d in data:
                print(f"{d[0]:<6} | {d[1]:<20} | {d[2]:<20} | {d[3]:<15} | {str(d[4]):<12} | {d[5]}")
    except Error as e:
        print("Could not load report:", e)
```

Figure 4 - Python code that I am proud of

This is the bubble sort in show_report(). The fetchall() returns tuples, so I converted them to a list. Two nested loops go through the data. The outer one tracks how many passes are left. The inner one compares each pair of rows by RegID. If the current ID is bigger than next one, the rows swap positions. In the end, the list is sorted from lowest to highest RegID.

APPENDIX 3 - PYTHON CODE SAMPLES SCREENSHOTS OF OOP

```python
# parent class for general user info
class User:  2 usages
    def __init__(self, user_id, name):
        self.user_id = user_id #encapsulated unique identifier
        self.name = name #combined full name

#StudentInfo class inheriting from the main User class
class StudentInfo(User):  2 usages
    def __init__(self, user_id, s_name, s_surname, s_email): #to set properties to StudentInfo
        self.s_name = s_name.strip()#strip() to clear accidental spaces from inputs
        self.s_surname = s_surname.strip()
        self.s_email = s_email
        #pass the id and combined full name up to the User class
        super().__init__(user_id,  name: f"{self.s_name} {self.s_surname}")

    def is_valid(self):  1 usage
        #name and surname format validation
        #checking that names contain only letters and be longer than 1 character
        if not (self.s_name.isalpha() and len(self.s_name) > 1):
            print(f"Validation error: Student name '{self.s_name}' is invalid")
            return False
        if not (self.s_surname.isalpha() and len(self.s_surname) > 1):
            print(f"Validation error: Student surname '{self.s_surname}' is invalid")
            return False

        #validating student email format
        if not ("@" in self.s_email and "." in self.s_email):
            print(f"Validation error: Email '{self.s_email}' is invalid")
            return False

        #if all check pass,capitalization before sending data to database
        self.s_name = self.s_name.capitalize()
        self.s_surname = self.s_surname.capitalize()
        return True
```

Figure 5 - StudentInfo Inheritance from User class

```python
#TeacherInfo class inheriting from the main User class
class TeacherInfo(User):  2 usages
    def __init__(self, user_id, t_name, t_surname, t_email, dept):
        self.t_name = t_name.strip()
        self.t_surname = t_surname.strip()
        self.t_email = t_email.strip()
        self.dept = dept.strip()
        super().__init__(user_id,  name: f"{self.t_name} {self.t_surname}")

    # name and surname format validation
    #checking that names contain only letters and be longer than 1 character
    def is_valid(self):  1 usage
        if not (self.t_name.isalpha() and len(self.t_name) > 1):
            print(f"Validation error: Teacher name '{self.t_name}' is invalid")
            return False
        if not (self.t_surname.isalpha() and len(self.t_surname) > 1):
            print(f"Validation error: Teacher surname '{self.t_surname}' is invalid")
            return False

        # validating teacher email format
        if not("@" in self.t_email and "." in self.t_email):
            print(f"Validation error: Email '{self.t_email}' is invalid")
            return False

        # capitalization
        self.t_name = self.t_name.capitalize()
        self.t_surname = self.t_surname.capitalize()
        self.dept = self.dept.capitalize()
        return True
```

Figure 6 - TeacherInfo Inheritance from User class

APPENDIX 4 - ENTITY RELATIONSHIP DIAGRAMS (CONCEPTUAL, LOGICAL, AND PHYSICAL)

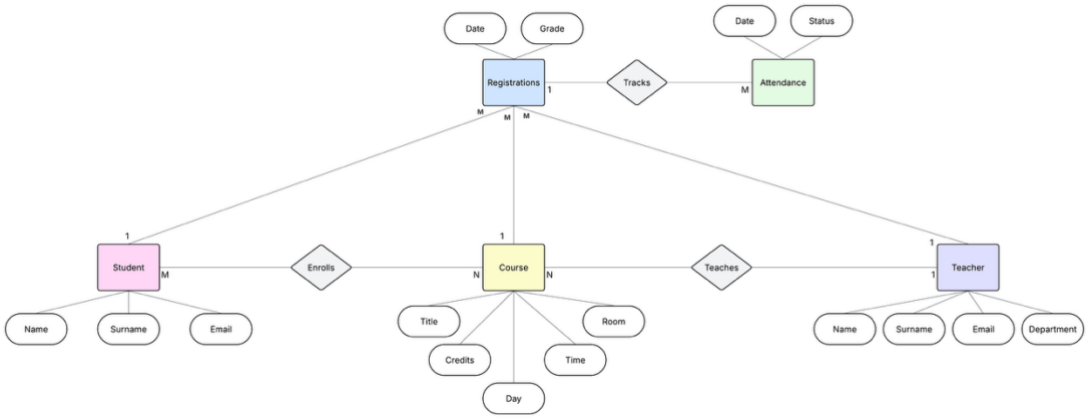


Figure 7 - Conceptual ERD

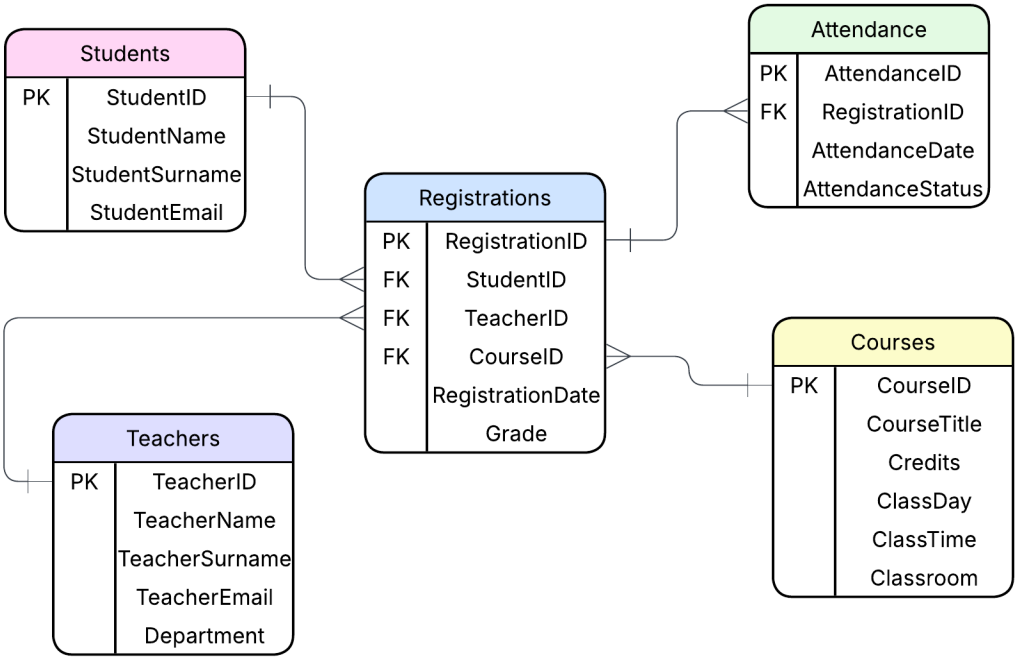


Figure 8 - Logical ERD

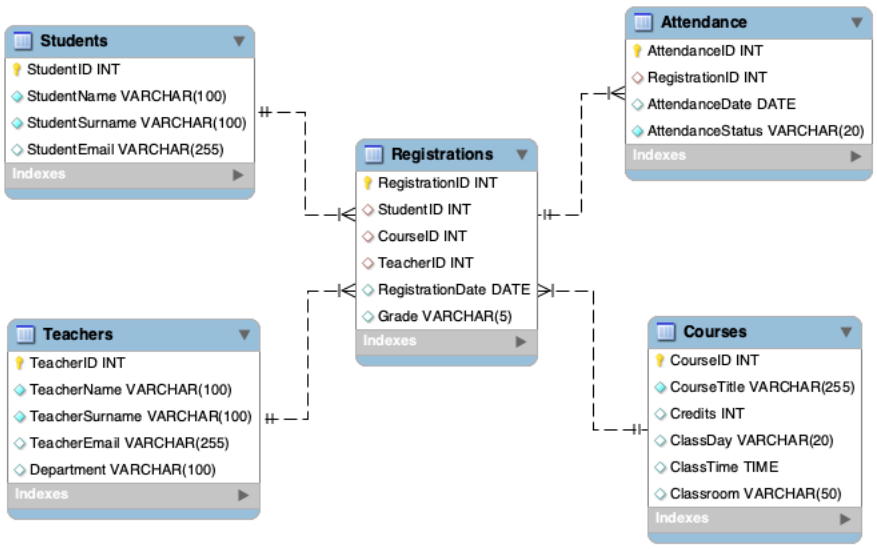


Figure 9 - Physical ERD

APPENDIX 5 – SAMPLES OF DDL/DML SQL STATEMENTS USED

```
5     -- Creating a table for student information
6     CREATE TABLE Students (
7         StudentID INT PRIMARY KEY,
8         StudentName VARCHAR(100) NOT NULL,
9         StudentSurname VARCHAR(100) NOT NULL,
10        StudentEmail VARCHAR(255) UNIQUE
11    );
12
13    CREATE INDEX idx_student_email ON Students(StudentEmail);
14
```

Figure 10 - CREATE TABLE Students statement and database index optimization

```
34      -- Creating registration table to link previous tables
35  ●   CREATE TABLE Registrations (
36          RegistrationID INT AUTO_INCREMENT PRIMARY KEY,
37          StudentID INT,
38          CourseID INT,
39          TeacherID INT,
40          RegistrationDate DATE,
41          Grade VARCHAR(5) DEFAULT 'N/A',
42          CONSTRAINT chk_grade CHECK (Grade IN ('A' , 'B', 'C', 'D', 'F', 'N/A')),
43          CONSTRAINT fk_student FOREIGN KEY (StudentID)
44              REFERENCES Students (StudentID)
45              ON DELETE CASCADE,
46          CONSTRAINT fk_course FOREIGN KEY (CourseID)
47              REFERENCES Courses (CourseID)
48              ON DELETE CASCADE,
49          CONSTRAINT fk_teacher FOREIGN KEY (TeacherID)
50              REFERENCES Teachers (TeacherID)
51              ON DELETE CASCADE
52      );
53
```

Figure 11 - CREATE TABLE Registrations statement

```
-- Creating table to check attendance
CREATE TABLE Attendance (
    AttendanceID INT AUTO_INCREMENT PRIMARY KEY,
    RegistrationID INT,
    AttendanceDate DATE,
    AttendanceStatus VARCHAR(20) NOT NULL,
    CONSTRAINT chk_status CHECK (AttendanceStatus IN ('Present' , 'Absent', 'Late')),
    FOREIGN KEY (RegistrationID)
        REFERENCES Registrations (RegistrationID)
        ON DELETE CASCADE
);
```

Figure 12 - CREATE TABLE Attendance statement

```
110     -- Creating a procedure to get all registrations with joined details
111 ●   CREATE PROCEDURE sp_GetAllRegistrations()
112     BEGIN
113         SELECT r.RegistrationID,
114                CONCAT(s.StudentName, ' ', s.StudentSurname) AS Student,
115                c.CourseTitle,
116                CONCAT(t.TeacherName, ' ', t.TeacherSurname) AS Teacher,
117                r.RegistrationDate, r.Grade
118         FROM Registrations r
119         JOIN Students s ON r.StudentID = s.StudentID
120         JOIN Courses c ON r.CourseID = c.CourseID
121         JOIN Teachers t ON r.TeacherID = t.TeacherID;
122     END //
```

Figure 13 - Stored procedure to get registration report

```
-- Creating a procedure to add a student
CREATE PROCEDURE sp_AddStudent(IN s_id INT, IN s_name VARCHAR(100), IN s_surname VARCHAR(100), IN s_email VARCHAR(255))
BEGIN
    INSERT INTO Students (StudentID, StudentName, StudentSurname, StudentEmail) VALUES (s_id, s_name, s_surname, s_email);
END //

-- Creating a procedure to update student details
CREATE PROCEDURE sp_UpdateStudent(IN s_id INT, IN s_name VARCHAR(100), IN s_surname VARCHAR(100), IN s_email VARCHAR(255))
BEGIN
    UPDATE Students SET StudentName = s_name, StudentSurname = s_surname, StudentEmail = s_email WHERE StudentID = s_id;
END //

-- Creating a procedure to add a teacher
CREATE PROCEDURE sp_AddTeacher(IN t_id INT, IN t_name VARCHAR(100), IN t_surname VARCHAR(100), IN t_email VARCHAR(255), IN t_dept VARCHAR(100))
BEGIN
    INSERT INTO Teachers (TeacherID, TeacherName, TeacherSurname, TeacherEmail, Department) VALUES (t_id, t_name, t_surname, t_email, t_dept);
END //

-- Creating a procedure to update teacher details
CREATE PROCEDURE sp_UpdateTeacher(IN t_id INT, IN t_name VARCHAR(100), IN t_surname VARCHAR(100), IN t_email VARCHAR(255), IN t_dept VARCHAR(100))
BEGIN
    UPDATE Teachers SET TeacherName = t_name, TeacherSurname = t_surname, TeacherEmail = t_email, Department = t_dept WHERE TeacherID = t_id;
END //

-- Creating a procedure to add a course
CREATE PROCEDURE sp_AddCourse(IN c_id INT, IN c_title VARCHAR(255), IN c_credits INT, IN c_day VARCHAR(20), IN c_time TIME, IN c_room VARCHAR(50))
BEGIN
    INSERT INTO Courses (CourseID, CourseTitle, Credits, ClassDay, ClassTime, Classroom) VALUES (c_id, c_title, c_credits, c_day, c_time, c_room);
END //

-- Creating a procedure to update course details
CREATE PROCEDURE sp_UpdateCourse(IN c_id INT, IN c_title VARCHAR(255), IN c_credits INT, IN c_day VARCHAR(20), IN c_time TIME, IN c_room VARCHAR(50))
BEGIN
    UPDATE Courses SET CourseTitle = c_title, Credits = c_credits, ClassDay = c_day, ClassTime = c_time, Classroom = c_room WHERE CourseID = c_id;
END //
```

Figure 14 - Stored Procedures for data Insertion and modification

```
-- Creating a procedure to get attendance report
CREATE PROCEDURE sp_GetAllAttendanceReport()
BEGIN
    SELECT CONCAT(s.StudentName, ' ', s.StudentSurname) AS StudentName,
           c.CourseTitle,
           a.AttendanceStatus
    FROM Attendance a
    JOIN Registrations r ON a.RegistrationID = r.RegistrationID
    JOIN Students s ON r.StudentID = s.StudentID
    JOIN Courses c ON r.CourseID = c.CourseID;
END //
```

Figure 15 - Stored Procedure to get attendance report

```sql
-- Setting grades
CALL sp_SetGrade(1, 'A');   CALL sp_SetGrade(2, 'B');   CALL sp_SetGrade(3, 'C');   CALL sp_SetGrade(4, 'A');
CALL sp_SetGrade(5, 'B');   CALL sp_SetGrade(6, 'A');   CALL sp_SetGrade(7, 'D');   CALL sp_SetGrade(8, 'C');
CALL sp_SetGrade(9, 'A');   CALL sp_SetGrade(10, 'B');  CALL sp_SetGrade(11, 'A');  CALL sp_SetGrade(12, 'A');
CALL sp_SetGrade(13, 'C');  CALL sp_SetGrade(14, 'B');  CALL sp_SetGrade(15, 'F');  CALL sp_SetGrade(16, 'A');
CALL sp_SetGrade(17, 'B');  CALL sp_SetGrade(18, 'C');  CALL sp_SetGrade(19, 'A');  CALL sp_SetGrade(20, 'B');

-- Marking attendance
CALL sp_MarkAttendance(1, '2026-05-21', 'Present');  CALL sp_MarkAttendance(2, '2026-05-21', 'Late');
CALL sp_MarkAttendance(3, '2026-05-21', 'Present');  CALL sp_MarkAttendance(4, '2026-05-21', 'Absent');
CALL sp_MarkAttendance(5, '2026-05-21', 'Present');  CALL sp_MarkAttendance(6, '2026-05-21', 'Present');
CALL sp_MarkAttendance(7, '2026-05-21', 'Late');     CALL sp_MarkAttendance(8, '2026-05-21', 'Present');
CALL sp_MarkAttendance(9, '2026-05-21', 'Absent');   CALL sp_MarkAttendance(10, '2026-05-21', 'Present');
CALL sp_MarkAttendance(11, '2026-05-21', 'Present'); CALL sp_MarkAttendance(12, '2026-05-21', 'Present');
CALL sp_MarkAttendance(13, '2026-05-21', 'Late');    CALL sp_MarkAttendance(14, '2026-05-21', 'Present');
CALL sp_MarkAttendance(15, '2026-05-21', 'Absent');  CALL sp_MarkAttendance(16, '2026-05-21', 'Present');
CALL sp_MarkAttendance(17, '2026-05-21', 'Present'); CALL sp_MarkAttendance(18, '2026-05-21', 'Late');
CALL sp_MarkAttendance(19, '2026-05-21', 'Present'); CALL sp_MarkAttendance(20, '2026-05-21', 'Present');

CALL sp_GetAllRegistrations();
CALL sp_GetAllAttendanceReport();


SELECT * FROM Students;
SELECT * FROM Teachers;
SELECT * FROM Courses;
SELECT * FROM Registrations;
SELECT * FROM Attendance;
```

Figure 16 - CALL statements from testing.sql

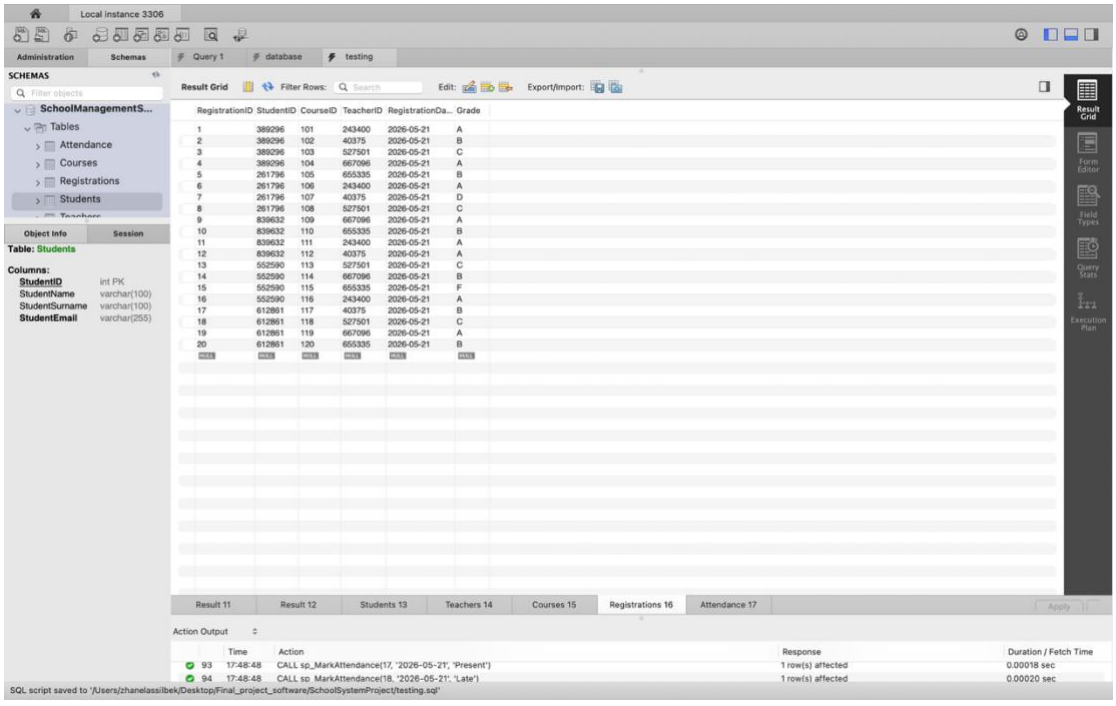


Figure 17 - Registration report in SQL database

APPENDIX 6 – TEST LOGS

Table 7: Test Logs

Test ID	Test type	What was tested	Used Input	Expected Result	Actual Result	Pass /Fail
T01	Manual	Add student with valid data	ID: 999, Name: Anna, Surname: Smith, Email: a@a.com	"Student Anna Smith was saved"	Message printed, record added to database	Pass
T02	Manual	Add student with non-numeric ID	ID: "abc"	ValueError caught, error message printed	"Input error: Student ID must be an integer numeric value."	Pass
T03	Manual	Add student with invalid email	Email: "notanemail"	Validation error, no DB call made	"Validation error: Email "notanemail" is invalid	Pass
T04	Manual	Add student with single-letter name	Name: "A"	Validation error, record not saved	"Validation error: Student name 'A' is invalid"	Pass
T05	Manual	Set invalid grade via set_grade()	Grade: "Z"	Grade rejected in db_controller.py before DB call	"Validation error: Grade 'Z' is invalid. Choose from : A, B, C, D, F, N/A"	Pass
T06	Manual	Find student with non-existent ID	ID: 00000	"Student not found" message	"Student with ID 0 not found."	Pass
T07	Manual	Delete student, check CASCADE	Delete StudentID linked to Registrations	Registrations and Attendance rows deleted automatically	Cascade worked, no orphaned records remained	Pass
T08	Integration	Run testing.sql script	All CALL statements executed sequentially	20 registrations inserted, grades and attendance set, reports printed	Both reports printed correctly with all 20 records	Pass
T09	Integration	CSV import with duplicate entries	Re-run import_data.py on same CSV file	Duplicates were skipped using ER_DUP_ENTRY, import continues	"Skipped duplicates" count printed, no crash	Pass
T10	Integration	Session variable NULL bug in testing.sql	Variables not initialised before SELECT INTO	Registrations linked to correct student and teacher IDs	Fixed by adding SET @s1 = 0 initialisers in testing.sql before SELECT statements	Pass

APPENDIX 7 – OUTPUT SCREENSHOTS



Figure 18 - Main menu

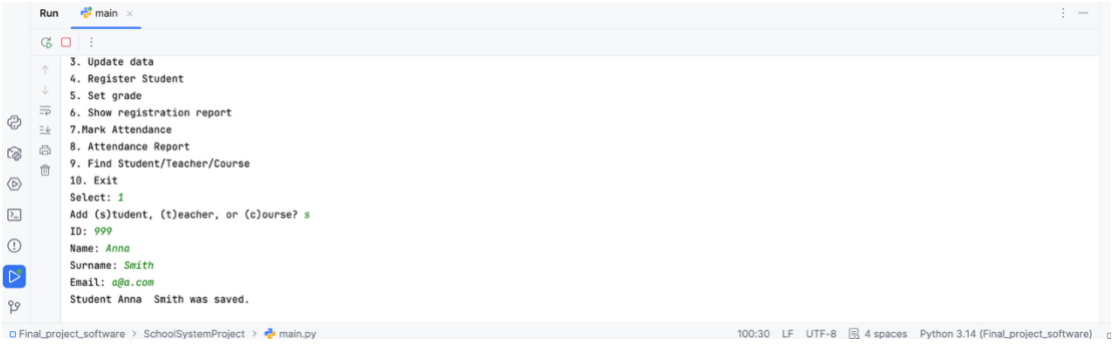


Figure 19 - Adding student with valid data

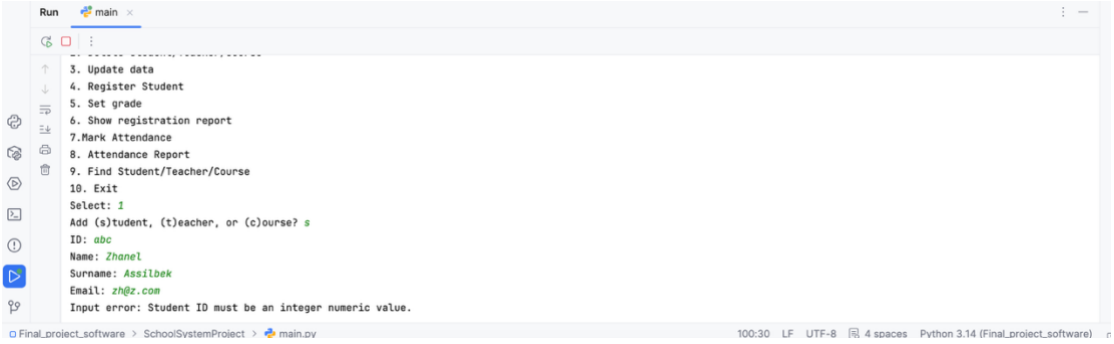


Figure 20 - Invalid ID testing

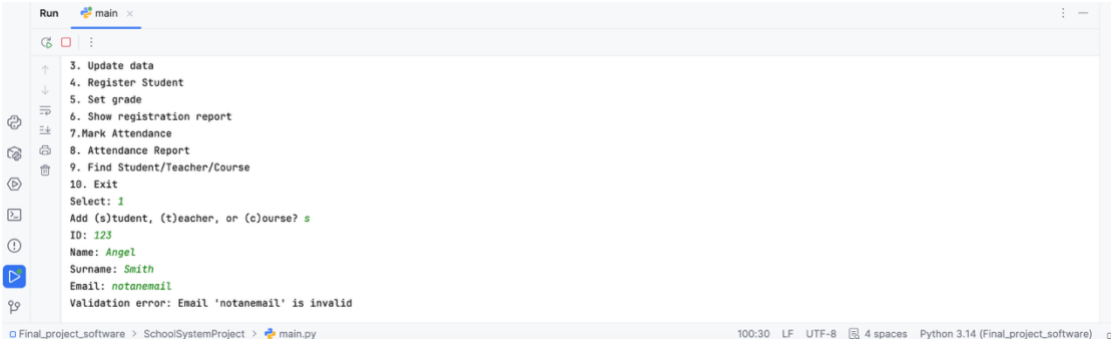


Figure 21 - Invalid email testing

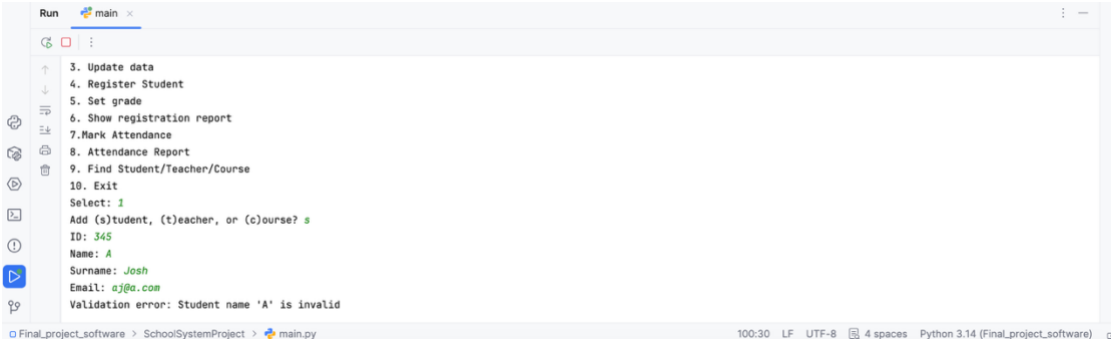


Figure 22 - Single-letter name testing



Figure 23 - Invalid grade

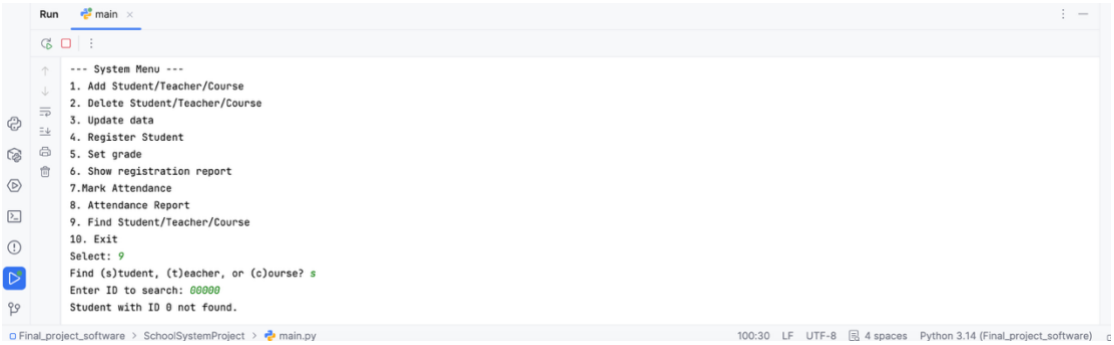


Figure 24 - Not existing ID testing

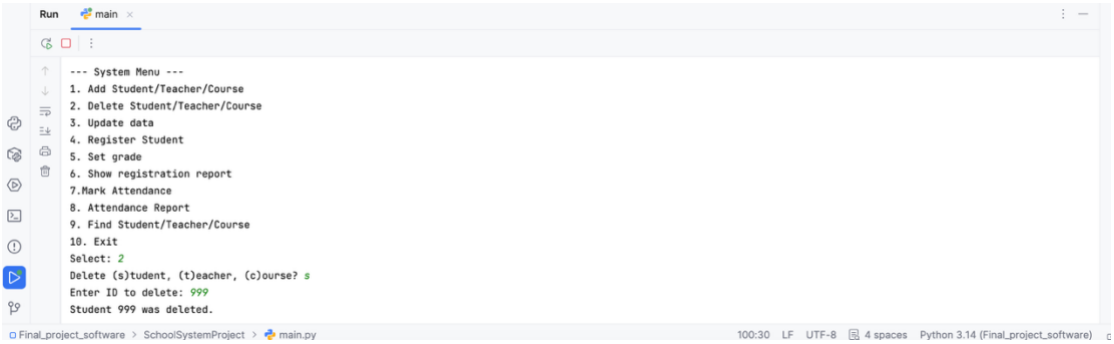


Figure 25 - Deleting student by ID



Figure 26 - Registration report



Figure 27 - Student Attendance Report